Astronomical Object Catalogue Project Report

Awwab Butt ID: 11091269

10/05/2025

Abstract

This project seeks to develop an astronomical object catalogue written in C++, utilizing OOP principles to organize and store data within a class hierarchy. It aims to offer a versatile platform for storing and managing astronomical data across different object types. Currently, it supports the 'base', star, galaxy, and planet types. Additionally, it facilitates the importing/exporting of data through formatted `.txt` files. It also includes a command line interface for viewing, editing, adding, exporting/importing, and generating reports on the stored data.

1 Introduction

Astronomy needs vast quantises of data to perform scientific work and as a result astronomers collect vast quantitates of data for countless objects across hundreds of surveys and experiments. Managing this data manually is a fool's errand, therefore the need arises for a system which can automatically manage the storing, ingesting and exporting of astronomical object data. This system should also provide a user friendly interface for convenient access to the information and be simple enough to be understood and used quickly even by people not well versed in programming.

The aim of this project is to create an easy to use astronomical object catalogue built by storing data in a class hierarchy, in which a specific object can be instantiated with some basic information and then observations about that object can be added. This data should be easily accessible easily and be easily exported as a formatted `.txt` for sharing with others in the astronomical community. It also aims to be as modular and extensible as possible so additions like adding more supported object types or adding modules which access and process the data is as easy as possible.

2 Implementation

At a high level the structure of the program is separated into three main components. The user interface, the catalogue interface and the back-end class based storage structure, with the catalogue acting as the intermediary between the UI and the actual data. The overall structure can be seen in Figure 1.

Data from observations are stored in encapsulated observation classes, corresponding to the observation type's need. These Observation classes have virtualised `print_observation` and `return_obs_formatted_file_string` methods which are used to print the observational data to the console, and return formatted csv string containing the data respectively.

Groups of these observations along with the astronomical object name and type are bound together in astronomical object classes. The observation data members explicitly included correspond to 'basic' observational data about each object type. For example a star's spectral

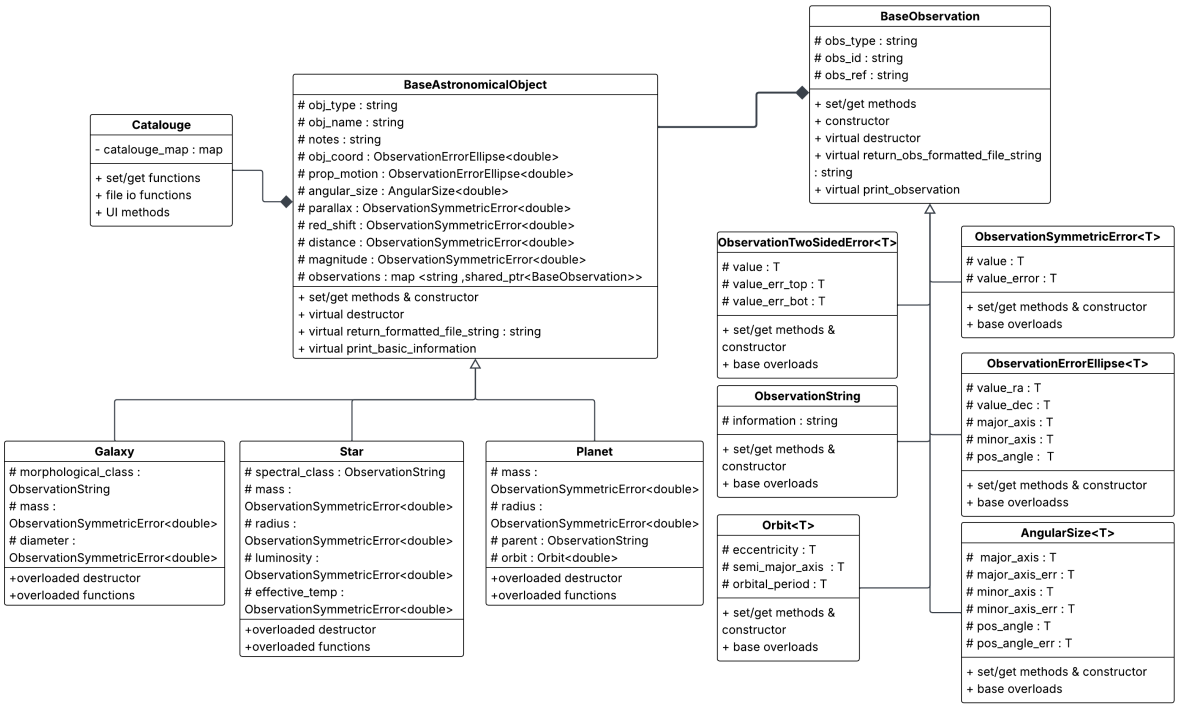

Figure 1: UML class structure diagram of the Catalogue.

class needs to be given as basic data in the `Star` class. Additional data beyond this 'basic' information is stored polymorphically in the `observations` [Advanced C++] `map` of the astronomical object. These are stored as `string` unique ID to `shared_ptr<BaseObservation>` pairs. Astronomical objects themselves have virtualised `print_all_infromation` and `return_formmated_file_string` methods which invoke the corresponding observation functions to print all the information to the console or return a formatted file string containing all the data respectively.

The `catalouge` class itself then stores these astronomical objects polymorphically within a central `string` object name to `shared_ptr<BaseAstronomicalObject> map`. When accessing objects `const` references are returned to avoid copying large objects where it is unnecessary i.e. just accessing a single observation.

Pointers are used in this data structure as they allow for dynamic memory allocation, which allows the program to accommodate large datasets without running out of memory or using it inefficiently in a massive preallocated stack. It also allows for these large objects to be passed and manipulated without expensive copying. The smart `shared_ptr` pointer was used as it automatically manages the memory, increasing memory safety as opposed to a raw pointer. Pointers also facilitate polymorphism which allows there to be a central storage structure containing all the data which we can can access with common overloaded methods without losing any data. The `obj_type`/`obs_type` data from the base classes can be used, if required, to facilitate safe downcasing to the appropriate derived classes to access the object data more granularly.

The `catalouge` class contains IO member methods for getting, deleting, printing data about astronomical objects and observations.

The `catalouge` class also has a `read_file` method which reads in data from formatted `.txt` files. It does this by reading the formatted file line by line and tokenizing it using the ',' delimiter. It then invokes the corresponding parser method to construct an object from the provided data. A `shared_ptr` of the object is then created and is polymorphically added to

the `catalouge_map`. Additional observations per object are stored in object name observation data pairs within the text file. The `catalouge_map` is checked to see if the named parent object exists and then adds the observation to it. It handles all of this in [Advanced C++] try catch loop and skips any lines it fails to parse, printing the line number and the caught exception as the reason why it failed. Custom exceptions, made using `std::runtime_error`, are used to present more information to the user about why and where it failed in the parser chain making it easier for the user to debug errors in the input file. The exact formatting required is given in section 5.

The `catalogue` also contains a `export_file` method which creates a `.txt` file and iterates through the `catalouge_map` invoking the formatted string methods, and appends them to the file.

2.1 Command Line Interface (CLI)

The second component of the catalogue is the command line interface (CLI) UI, which is the main method the end user interacts with the catalogue. It is started from `open_cli_interface` member method of the catalogue. It works in a loop which waits for the user input, reads the input line from the input stream, and then tokenizes the input. This tokenized input is parsed by if statements to find the command it's trying to invoke and then runs it, after which the loop repeats. If the command is not recognised it prints out an error to console.

A 'selection' `vector<string>` is held in memory when the CLI runs. This allows the user to specify a selection of the objects to run various commands on, for example exporting a subset of the catalogue. It does this by storing the keys (names) of the objects it acts on. It also persists between command loops unless actively cleared by the user. Objects can be added by name or type and be removed from the selection. Functions are passed these selection keys which are used to find the objects they act on.

2.2 Mollweide projection [Additional functionality]

A method of visualising positional astronomical data was desired so a Mollweide projection was chosen. The Mollweide projection projects the 3D celestial sphere surface onto a 2D plane [1]. It is often used in astronomical visualisation most famously in the various CMB maps. Although all projections are imperfect mappings its traits like the equal area preserving property makes it good in astronomical visualisation. It works by mapping the longitude and latitude of positions on a spherical surface onto the x-y plane with the following mapping:

$$x = \frac{2\sqrt{2}}{\pi}(\lambda - \lambda_0)\cos\theta \tag{1}$$

$$y = R\sqrt{2}\sin\theta \tag{2}$$

where θ can be calculated using

$$2\theta + \sin 2\theta - \pi \sin\varphi = 0. \tag{3}$$

Where R is the radius of the sphere being projected, λ is the longitude, λ_0 is the meridian and φ is the latitude. For the purposes of this projection R was set to 1, and the meridian was at 0. To solve for θ a [Advanced C++] templated Newton-Raphson solver (Figure 2.2) with `std::function` pointers to arbitrary functions was created. This was done so that it could be reused for other visualisations or data processing in future iterations of the program. Equation 3 and its derivative were passed as [Advanced C++] lambda functions and theta was solved for. The Mollweide mapping was implemented such that it returned [Advanced C++]

`std::pairs` of x,y value pairs for easier handling. A pixel array for the base ellipse was then generated and grid lines corresponding to 15^o right-ascension/declination steps were overlaid. Then astronomical data was read from the `catalouge`, and their coordinates projected on the base ellipse with colours corresponding to the object type, with base being white, star yellow, galaxy purple and planet green. This pixel data was then exported as a `.bmp` image file.

```
template<typename T> T newton_raphson(std::function<T(T)> f,
std::function<T(T)> df, T guess, T tolerance, int max_iteration, T epsilon)
{
  T x = guess; // guess is the intial value
  for (size_t i = 0; i < max_iteration; i++)
  {
    T y = f(x);
    T y_prime = df(x);
    T x1 = x - (y / y_prime);
    if (abs(y_prime) < epsilon)
    {
      break;
    }
    if (std::abs(x1 - x) < tolerance)
    {
      return x1;
    }
    x = x1;
  }
  return x;
}

static double solve_theta(double lat)
{
  /*static becasue it's a common function name and only want within this file scope*/
  double delta = 1e-9;
  int iterations = 100;
  double theta = lat;
  theta = newton_raphson<double>([lat](double theta) -> double { return
  (2*theta+std::sin(2*theta)-pi * std::sin(lat)); },[](double theta)
  -> double {return(2+2*std::cos(2 * theta));},theta, delta, iterations, 1e-9);
  return theta;
}


std::pair<double, double> forward_mollweide(double lat, double longn, double radius, double meridian)
{
  double theta = solve_theta(lat);
  double x = radius * ((2 * std::sqrt(2)) / (pi)) * (longn - meridian) * std::cos(theta);
  double y = radius * (std::sqrt(2)) * std::sin(theta);
  return {x, y};
}
```

Figure 2: Newton-Raphson template function, auxiliary angle solver and the forward Mollweide function.

3 Observation Classes

3.1 BaseObservation

The base observation class, although not an abstract class, is only meant to be inherited from and not instantiated. All of the observation classes are derived from this. It contains `obs_type` which specifies the observation type, `obs_id` which is the unique identifier and `obs_ref` which contains the Bibcode reference to the data source. The bibcode standard was used as this is used across many databases as it is the standard way of querying the NASA Astrophysics Data System which acts as central repository for astronomical papers. If there is non references the catalogue accepts the shorthand `n/a`.

The base class also has two virtual member functions which are overloaded by all the derived classes `print_observation` and `return_obs_formatted_file_string`, they print the observational data to the console and return a formatted string for export respectively.

All the derived classes if possible are [Advanced C++] templated such that if the data type needs to be changed to increase precision it can be easily done. In the current program observations are instantiated as `double` types as it was found to be a good middle ground between required precision and size.

3.2 ObservationSymmetricError

The `ObservationSymmetricError` class is for data with a symmetric error bounds like the redshift. It holds a value and its error.

3.3 ObservationTwoSidedError

The `ObservationTwoSidedError` class is for data with two sided error bounds. It holds the value, the top error bound and the bottom bound. Observations like the distance are typically found two error bounds.

3.4 ObservationErrorEllipse

The `ObservationErrorEllipse` class holds data for observations with error ellipses like the celestial coordinates. It holds the right ascension and declination components and defines an error ellipse with the major axis, minor axis and the position angle.

3.5 ObservationString

The `ObservationString` class holds data that is just a string like the spectral class of a stellar object. It just has an additional string data member to hold this information.

3.6 Orbit

The `Orbit` class holds basic orbital parameters of an object. It holds the semi-major axis, eccentricity and the orbital period.

3.7 AngularSize

The `AngularSize` class holds data about the angular size of an object. Although this is a very specialised observation type and could be made by nesting the more primitive classes it was decided that nesting would cause confusion. The angular size is held as an ellipse with errors, it holds the major-axis and its error, the minor axis and the error and the position angle and its error.

4 Astronomical object classes

4.1 BaseAstronomicalObject

The `BaseAstronomicalObject` is the base class from which all the astronomical objects are derived. The derivation chain can be seen in Figure 1. The base astronomical object contains basic information on the object it's type, name, coordinates, the proper motion, angular size, parallax, redshift, distance, magnitude, notes and additional observations in `observations`. These 'basic' observations were decided to form a complete minimal set of information to describe an arbitrary astronomical object. From this base data other more specialised object types can be derived easily.

4.2 Star

The `Star` class contains additional about the star's spectral class, its mass, radius, luminosity and the effective black body temperature. Spectral classifications like the Morgan-Keenan are used to describe and differentiate stellar objects in succinct manner.

4.3 Galaxy

The `Galaxy` class contains additional information about the galaxy's morphological class, mass and diameter. The morphological class is a standard used by astronomers to describe the shape of galaxies succinctly.

4.4 Planet

The `Planet` contains additional information about the the planet's parent object, its orbit, mass and radius. The orbit and parent object are needed to completely describe the planet since all the current ways to find exoplanets rely on these parameters like measuring the transit.

5 Data Formatting

5.1 Observations

The fundamental observation object is the `BaseObservation` class it is stored in a comma seperated formatted as shown in table 1. Derived object classes use the same format and then append their additions at the end. For example the combined `ObservationSymmetricError` csv format would be obs_type,obs_id,obs_ref,value,error. Additional observations are stored by added the object name then the observation data section, obj_name,[rest of data]. These extra observations must be added after the line the fundamental object is defined else the parser will reject them. T corresponds to the template type, however currently they are parsed using the double data type.

Table 1: BaseObservation csv format

obs_type	obs_id	obs_ref
string	string	string

Table 2: ObservationSymmetricError csv format

value	error
T	T

Table 3: ObservationErrorEllipse csv format

Right ascension	Declination	Major axis	Minor axis	Position angle
T	T	T	T	T

Table 4: ObservationString csv format

Information
string

Table 5: ObservationTwoSidedError csv format

Value	Top error bound	Bottom error bound
T	T	T

Table 6: Orbit csv format

Eccentricity	Semi-major axis	Orbital period
T	T	T

Table 7: AngularSize csv format

Major-axis	Major error	Minor axis	Minor error	Position angle	PA error
T	T	T	T	T	T

5.2 Observation Units

The parser, input validation and units are currently tied to the observation type. As such there are currently a limited number of supported observation types. Table 8 in the appendix lists the supported observation type, `obs_type` string that is checked for, the observation class it is stored in and then the units expected.

5.3 Astronomical Objects

Astronomical objects are formed from a set of mandatory 'basic' observational data, the object name and type. They are stored in the following format, object type,object name,[basic observations csv subsections]. These observation sections are formatted as shown in Section 5.1 except the immanently follow one another on the same line. After the object type and name the order of the basic data sub sections don't matter, only that they are all present and parsable. Notes are a special type they are parsed by looking for the `notes` string and copy the token immediately after. Example objects are available in the `test_datasets` directory.

6 Usage guide

Once compiled the program runs in the console, and then waits for input commands. A comprehensive guide on the commands and how to use them is given in the program by typing in `help` once prompted into the console. Test data files are provided in the `test_datasets` folder.

7 Results

When the program ran the CLI opens automatically opens. Test Data has been provided in `test_datasets/data.txt`. Calling the read command on `data.txt` should produce the output seen in Figure 3, the errors seen are intentional to test the error handling mechanisms. Typing `print sirius` will print all the information about Sirius it should output Figure 4. Performing the `projection` command on all the data from the `test_datasets/data.txt` dataset should generate the projection seen in Figure 5.

```
Failed to read line 2
Reason: no_valid_parse_for_string_provided_check_it_follows_the_correct_format
Failed to read line 3
Reason: parent_object_doesnt_exist_for_observation
Completed reading test_datasets/data.txt file
```

Figure 3: Read data.txt test.

```
print sirius
|----------------------------------------------------------------------|
star sirius basic information, has 2 extra observations :

coordinates Data | Observation ID: 1 | Reference: 2007A&A...474..653V
Right ascension(deg): 1.012872e+02 | Declination(deg): -1.671612e+01 | Error ellipse | Major-axis(mas): 1.170000e+01 | Minor-axis(mas): 1.090000e+01 | Position angle(deg): 9.000000e+01

proper_motion Data | Observation ID: 2 | Reference: 2007A&A...474..653V
Right ascension(mas/yr): -5.460100e+02 | Declination(mas/yr): -1.223070e+03 | Error ellipse | Major-axis(mas): 1.330000e+00 | Minor-axis(mas): 1.240000e+00 | Position angle(deg): 0.000000e+00

angular_size Data | Observation ID: 3 | Reference: n/a
Major-axis(mas): 0.000000e+00 | +/-: 0.000000e+00 | Minor-axis(mas): 0.000000e+00 | +/-: 0.000000e+00 | Position angle(deg): 0.000000e+00 | +/-: 0.000000e+00

parallax Data | Observation ID: 4 | Reference: 2007A&A...474..653V
Value(mas): 3.792100e+02 | +/-: 1.580000e+00

redshift Data | Observation ID: 5 | Reference: 2006AstL...32..759G
Value(): 1.800000e-05 | +/-: 1.000000e-06

distance Data | Observation ID: 6 | Reference: 2007A&A...474..653V
Value(ly): 8.600000e+00 | +: 4.000000e-02 | -: 4.000000e-02

magnitude Data | Observation ID: 7 | Reference: 2002yCat.2237....0D
Value(): -1.510000e+00 | +/-: 0.000000e+00

Notes: some very interesting notes about sirius

spectral_class Data | Observation ID: 1231231 | Reference: 2003AJ....126.2048G
Information: A0mA1 Va

mass Data | Observation ID: 10 | Reference: 2017ApJ...840...70B
Value(kg): 4.103140e+30 | +/-: 2.300000e-02

radius Data | Observation ID: 9 | Reference: 2017ApJ...840...70B
Value(m): 1.190000e+09 | +/-: 9.000000e-03

luminosity Data | Observation ID: 11 | Reference: 2017ApJ...840...70B
Value(Lo): 2.470000e+01 | +/-: 7.000000e-01

effectivetemp Data | Observation ID: 12 | Reference: 2017ApJ...840...70B
Value(K): 9.845000e+03 | +/-: 6.400000e+01

There are 2 observations
coordinates Data | Observation ID: 1 | Reference: 2007A&A...474..653V
Right ascension(deg): 1.012872e+02 | Declination(deg): -1.671612e+01 | Error ellipse | Major-axis(mas): 1.170000e+01 | Minor-axis(mas): 1.090000e+01 | Position angle(deg): 9.000000e+01

parallax Data | Observation ID: 4 | Reference: 2007A&A...474..653V
Value(mas): 3.792100e+02 | +/-: 1.580000e+00
```

Figure 4: Print information test for the Star object 'Sirius'.

8 Discussion

Overall I believe this project creates a strong extensible foundation for cataloguing astronomical objects, however there are clear areas where it could be improved.

Currently observation units are hardcoded, this results in readability/usability issues like quoting stellar masses in kg rather than the more comprehensible solar masses. In a future update the observation data should allow for the units of the measurement to be specified.

Another pain point is that in astronomy the same objects in different surveys are referred to by different names, i.e. Messier 31 and Andromeda. Therefore a way to disambiguate them

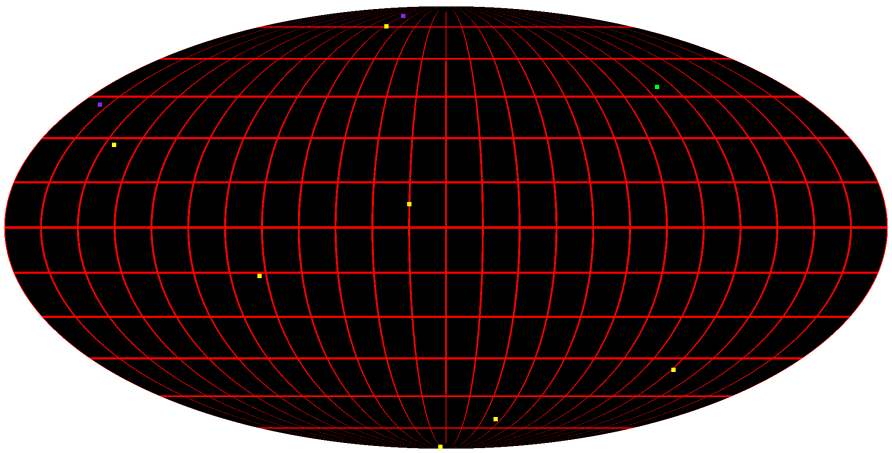

Figure 5: Expected Mollweide projection using the data from test_datasets/data.txt.

is needed so that data can be collected across multiple datasets. Typically this is done by comparing coordinate data or by referring to a central database containing the aliases like SIMBAD [2]. The central map, name to pointer pair data structure lends itself well to adding aliases to the same object without data duplication, so adding the functionality would be relatively easy. This was initially intended but time constraints lead to it being cut in this version.

9 Acknowledgement

The design of this catalogue was inspired by the designs of other astronomical object databases like SIMBAD, the NASA/IPAC Extragalactic Database and the Exoplanet Archive.

Latex word count: 2448.

References

[1] Miljenko Lapaine. "Mollweide map projection". In: *KoG* (Jan. 2011), pp. 7–16.

[2] M. Wenger et al. "The SIMBAD astronomical database. The CDS reference database for astronomical objects". In: 143 (Apr. 2000), pp. 9–22. DOI: 10.1051/aas:2000332.

A Parser table

Table 8: Observation type parser list

Observation type	obs_type	Observation class	Units
Celestial coordinates	coordinates	ObservationErrorEllipse	See specific
Celestial coordinates :: Right ascension	coordinates	ObservationErrorEllipse	Degrees
Celestial coordinates :: Declination	coordinates	ObservationErrorEllipse	Degrees
Celestial coordinates :: Position angle	coordinates	ObservationErrorEllipse	Degrees
Celestial coordinates :: Major axis	coordinates	ObservationErrorEllipse	mas
Celestial coordinates :: Minor axis	coordinates	ObservationErrorEllipse	mas
Distance	distance	ObservationTwoSidedError	Light years
Proper motion	proper_motion	ObservationErrorEllipse	See specific
Proper motion :: Right ascension	proper_motion	ObservationErrorEllipse	mas/yr
Proper motion :: Declination	proper_motion	ObservationErrorEllipse	mas/yr
Proper motion :: Major axis	proper_motion	ObservationErrorEllipse	mas/yr
Proper motion :: Minor axis	proper_motion	ObservationErrorEllipse	mas/yr
Proper motion :: Position angle	proper_motion	ObservationErrorEllipse	Degrees
Parallax	parallax	ObservationSymmetricError	mas
Orbit	orbit	Orbit	See specificd
Orbit :: Semi-major axis	orbit	Orbit	meters
Orbit :: Orbital period	orbit	Orbit	solar days
Parent orbital object	parent	ObservationString	N/A
Redshift	redshift	ObservationSymmetricError	Unitless
Brightness magnitude	magnitude	ObservationSymmetricError	Unitless
Apparent angular size	angular_size	AngularSize	mas
Radius	radius	ObservationSymmetricError	meters
Mass	mass	ObservationSymmetricError	kg
Effective Blackbody temperature	effectivetemp	ObservationSymmetricError	Kevlin
Luminosity	luminosity	ObservationSymmetricError	Solar luminosity
Stellar spectral classification	spectral_class	ObservationString	N/A
Diameter	diameter	ObservationSymmetricError	meters
Galactic Morphological class	morphological_class	ObservationString	N/A